Designmodell

Modul: T4INF2003 – Software Engineering I
Kurs: TINF2024
Dozentin: Charlotte Chichlow
Semester: Frühjahr 2026

Projekt: Entwicklung eines Online Aufgaben-Planungssystems

Designmodell....1
1. Überblick und Zielsetzung....2
2. Systemübersicht und Architektur....2
    2.1 Produktbeschreibung....2
    2.2 Architekturentscheidung: Client-Server....2
    2.3 Schichten und Technologieentscheidungen....3
3. Statisches Modell....3
    3.1 Paketstruktur....3
    3.2 Klassendiagramm....6
        3.2.1 ER-Schema der Datenbank....7
    3.3 Architektur / Services....9
4. Dynamisches Modell....10
    4.1 Use-Case-Diagramm....10
    4.2 Sequenzdiagramm: Benutzer registrieren und anmelden....11
    4.3 Sequenzdiagramm: Aufgabe erstellen....11
    4.4 Sequenzdiagramm: Arbeitsplan generieren....12
        4.4.1 Beschreibung des Planungsalgorithmus....12
    4.5 Sequenzdiagramm: Aufgabe bearbeiten....14
    4.6 Identifizierte Methoden je Klasse....14
5. API-Spezifikation....15
6. Design Patterns....16
7. Frontend-Struktur....17
8. Scope der ersten Iteration (MVP)....18
9. Deployment und Entwicklungsinfrastruktur....18

1. Überblick und Zielsetzung

Das System „TeaPot" ist eine browserbasierte Webanwendung, die Einzelpersonen und Teams dabei unterstützt, Aufgaben nicht nur zu erfassen, sondern auch automatisiert und ressourcenbasiert zu planen. Im Gegensatz zu klassischen To-Do-Listen oder Kalender Applikationen überbrückt TeaPot die Lücke zwischen statischem Aufgabenmanagement und aktivem Kalendermanagement: Ein Planungsalgorithmus legt Aufgaben anhand von Priorität, Deadline, Abhängigkeiten und individuellem Arbeitsprofil selbstständig in freie Zeitfenster ein.

Die Designstudie hat folgende Ziele:

- Validierung der gewählten Technologien (React, C#, SQL Server)
- Prototypische Umsetzung zentraler Geschäftslogik
- Sammlung von Erfahrungen mit der geplanten Architektur
- Früherkennung von Risiken und Herausforderungen

2. Systemübersicht und Architektur

2.1 Produktbeschreibung

Das System richtet sich an Einzelpersonen und Teams, die Aufgaben strukturiert planen und koordinieren wollen. Benutzer können Aufgaben mit Prioritäten (Low, Medium, High), Deadlines, Zeitschätzungen und Abhängigkeiten konfigurieren sowie über ein persönliches Arbeitsprofil (tägliche Arbeitszeit, maximale Belastung) die Rahmenbedingungen für einen automatisch generierten Arbeitsplan definieren.

2.2 Architekturentscheidung: Client-Server

Das System setzt auf eine **drei-schichtige Client-Server-Architektur** mit klarer Trennung von Frontend (React), Backend (ASP.NET Core / C#) und Datenpersistenz (SQL Server). Die Kommunikation zwischen Frontend und Backend erfolgt ausschließlich über eine REST-API. Diese Architektur wurde gewählt, weil:

- **Separation of Concerns**: Frontend und Backend können unabhängig voneinander entwickelt, getestet und gewartet werden.
- **Skalierbarkeit**: Mehrere Clients können denselben Backend-Server nutzen, ohne dass Änderungen am Server notwendig werden.
- **Sicherheit**: Authentifizierung (JWT-Token) und Autorisierung werden zentral auf dem Server verwaltet.
- **Testbarkeit**: Backend-Logik und Frontend-Komponenten lassen sich isoliert testen.

2.3 Schichten und Technologieentscheidungen

Schicht	Technologie	Begründung
Frontend	React 18+	Wiederverwendbare Komponenten, Virtual DOM, breite Community
Backend	C# 11+ / ASP.NET Core 8+	Typsicherheit, async/await, Dependency Injection, LINQ
Datenbank	SQL Server 2019+	ACID-Compliance, EF-Core-Integration, Skalierbarkeit
API-Kommunikation	REST (JSON)	Standardisiert, gut unterstützt, einfach testbar
ORM	Entity Framework Core	Abstrahiert SQL, ermöglicht saubere Repository-Schicht
Authentifizierung	JWT (JSON Web Token)	Zustandslos, skalierbar, gut für SPAs geeignet

3. Statisches Modell

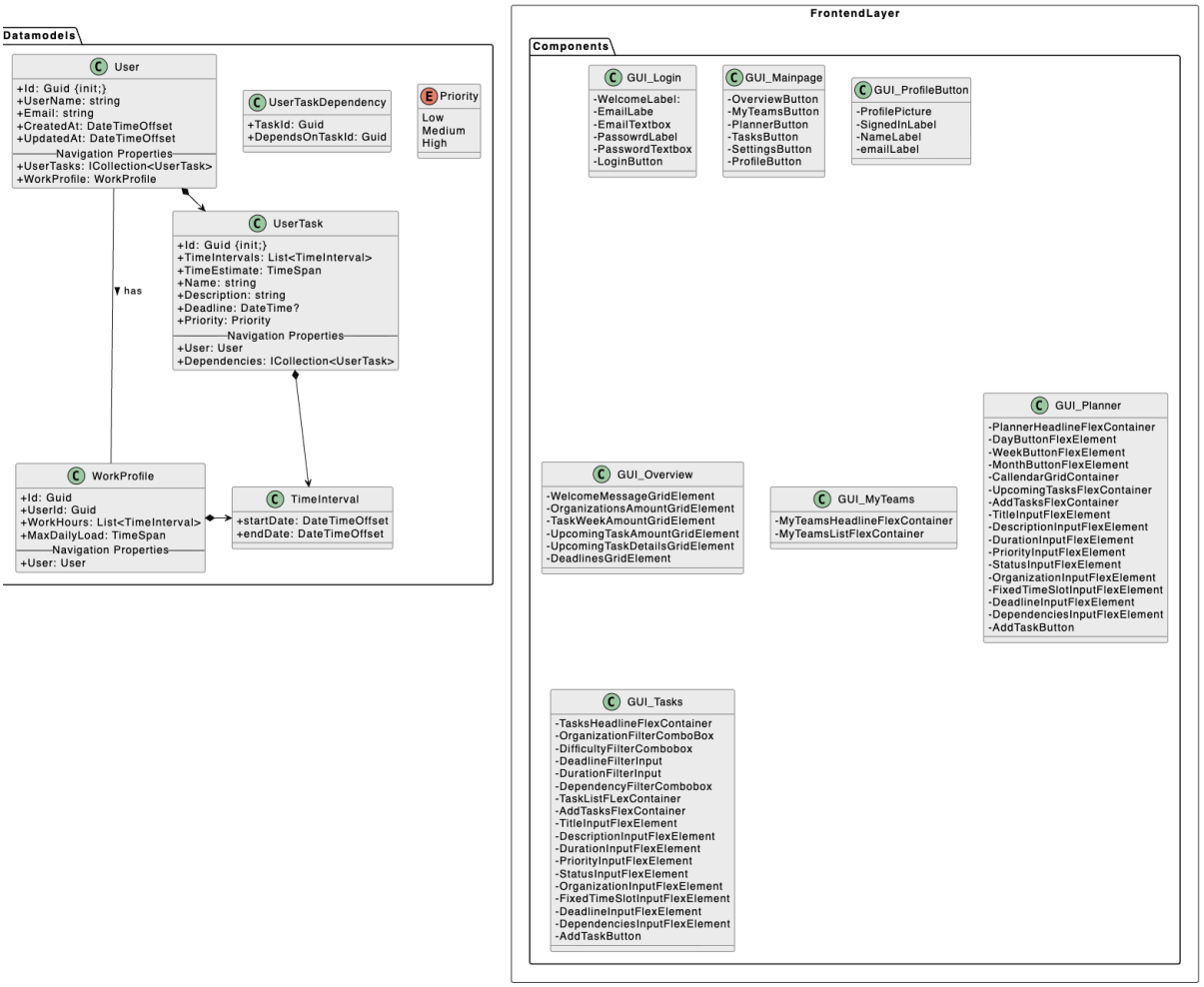
Hinweis zum Scope: In der ersten Iteration des Designmodells werden nur die für den MVP relevanten Kernobjekte (User, UserTask, WorkProfile, TimeInterval) detailliert modelliert und in den Sequenzdiagrammen verwendet. Die im Pflichtenheft definierten Objekte **Organisation**, **Arbeitsplan**, **Aufgabenblock** und deren Assoziationen sind fachlich zentral, werden aber für Iteration 1 nur im statischen Modell ergänzt, um die spätere Erweiterung zu ermöglichen. Im Fokus stehen zunächst die reinen Nutzer- und Planungsfunktionen.

3.1 Paketstruktur

Die Darstellung erfolgt aus Gründen der Übersichtlichkeit in mehreren Teilabbildungen.

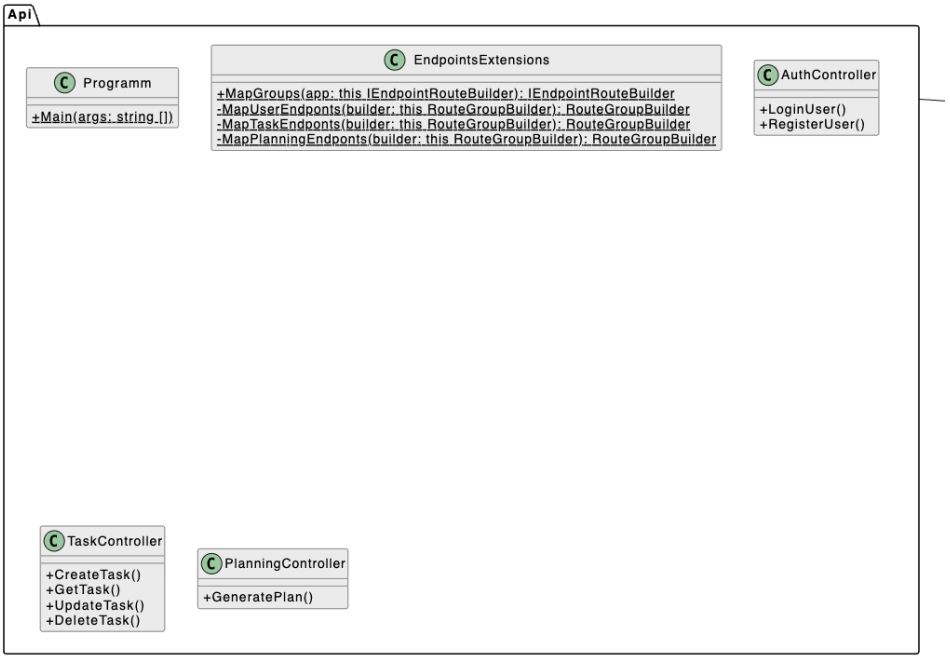
Datamodels und Frontend

Die folgende Abbildung zeigt das Paket „Datamodels“ sowie Teile des FrontendLayer.
Das Paket „Datamodels“ enthält die zentralen Entitäten des Systems, wie User, UserTask, WorkProfile und TimeInterval.
Diese bilden die Grundlage für die Datenpersistenz und modellieren die fachlichen Objekte der Anwendung.
Der FrontendLayer enthält die React-Komponenten, die für die Benutzeroberfläche verantwortlich sind.
Sie ermöglichen die Interaktion der Nutzenden mit dem System und bilden die Präsentationsschicht der Anwendung.



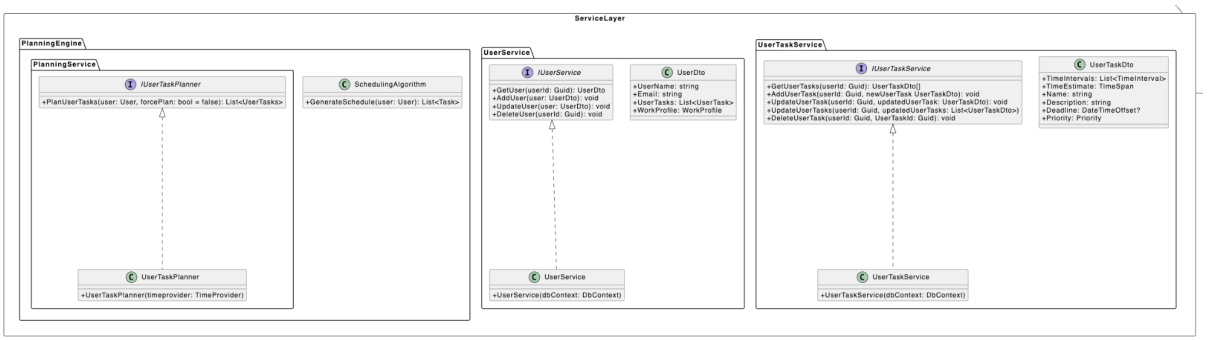
API-Paket

Die folgende Abbildung zeigt das API-Paket des Systems.
Dieses stellt die Schnittstelle zwischen Frontend und Backend dar.
Die Controller verarbeiten eingehende HTTP-Anfragen und delegieren die Geschäftslogik an die entsprechenden Services.
Dadurch wird eine klare Trennung zwischen Präsentationslogik und Geschäftslogik erreicht.



ServiceLayer und PlanningEngine

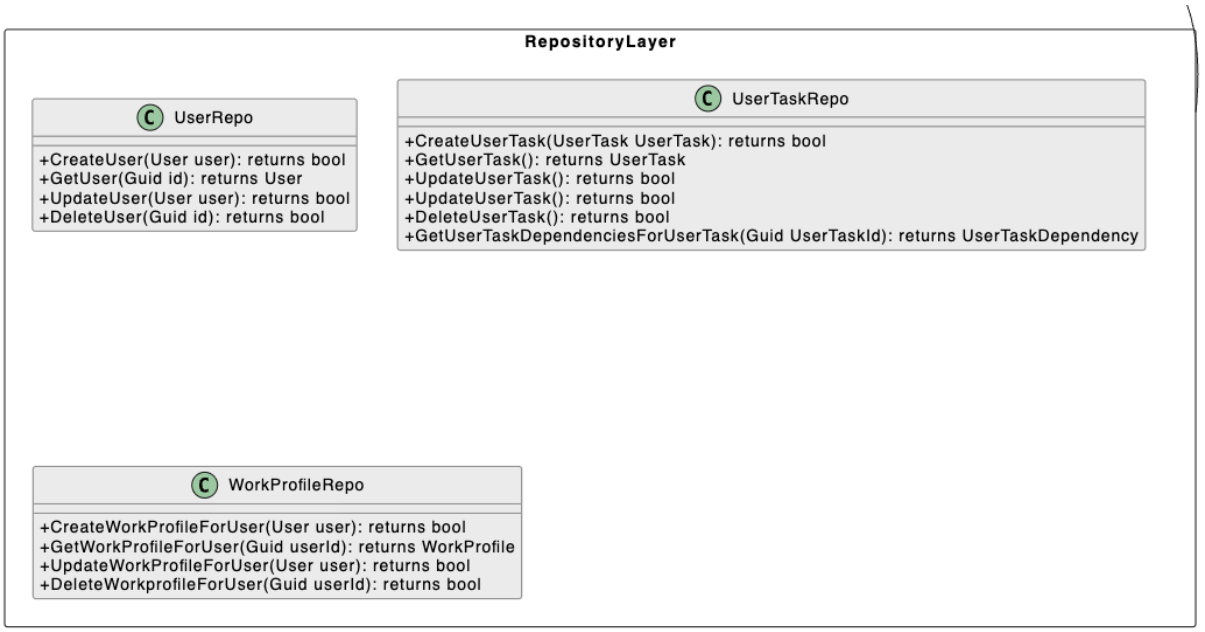
Die folgende Abbildung zeigt das ServiceLayer-Paket sowie die PlanningEngine.
Der ServiceLayer kapselt die Geschäftslogik des Systems und stellt definierte Schnittstellen für die Verarbeitung von Benutzern und Aufgaben bereit.
Die PlanningEngine ist als eigenständiges Subsystem implementiert und enthält den Planungsalgorithmus.
Der SchedulingAlgorithm berechnet auf Basis von Prioritäten, Deadlines, Abhängigkeiten und benutzerdefinierten Einschränkungen einen Arbeitsplan.
Diese Trennung ermöglicht eine bessere Wartbarkeit sowie den Austausch des Algorithmus (Strategy Pattern).



RepositoryLayer

Die folgende Abbildung zeigt das RepositoryLayer-Paket.
Dieses übernimmt den Zugriff auf die Datenbank und kapselt alle Datenbankoperationen.
Die Repositories stellen Methoden zur Verfügung, um Entitäten zu speichern, zu laden, zu aktualisieren und zu löschen.

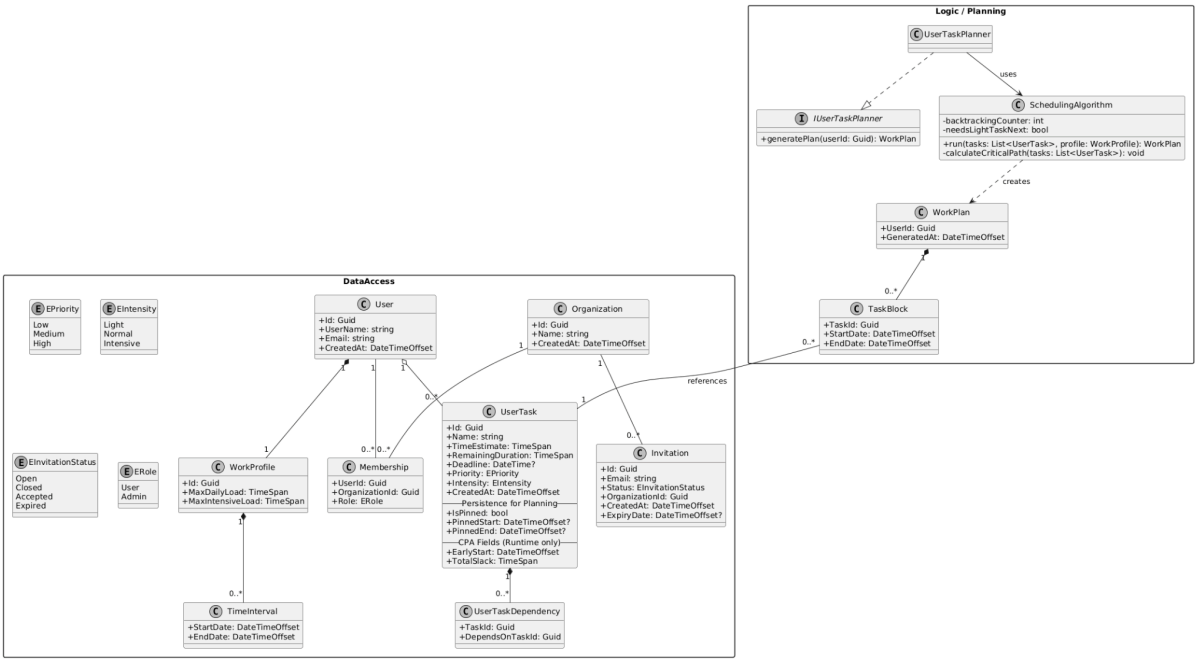
Durch diese Abstraktion wird der Zugriff auf die Daten von der Geschäftslogik getrennt, was die Testbarkeit und Wartbarkeit des Systems verbessert (Repository Pattern).



3.2 Klassendiagramm

Das dargestellte Klassendiagramm bildet die Brücke zwischen der statischen Datenhaltung im Paket DataAccess und der dynamischen Planungslogik in Logic / Planning. Im Zentrum steht die Erweiterung der UserTask-Entität um spezifische Parameter der *Critical Path Analysis* (CPA) sowie Persistenz-Attribute für fixierte Aufgabenblöcke .

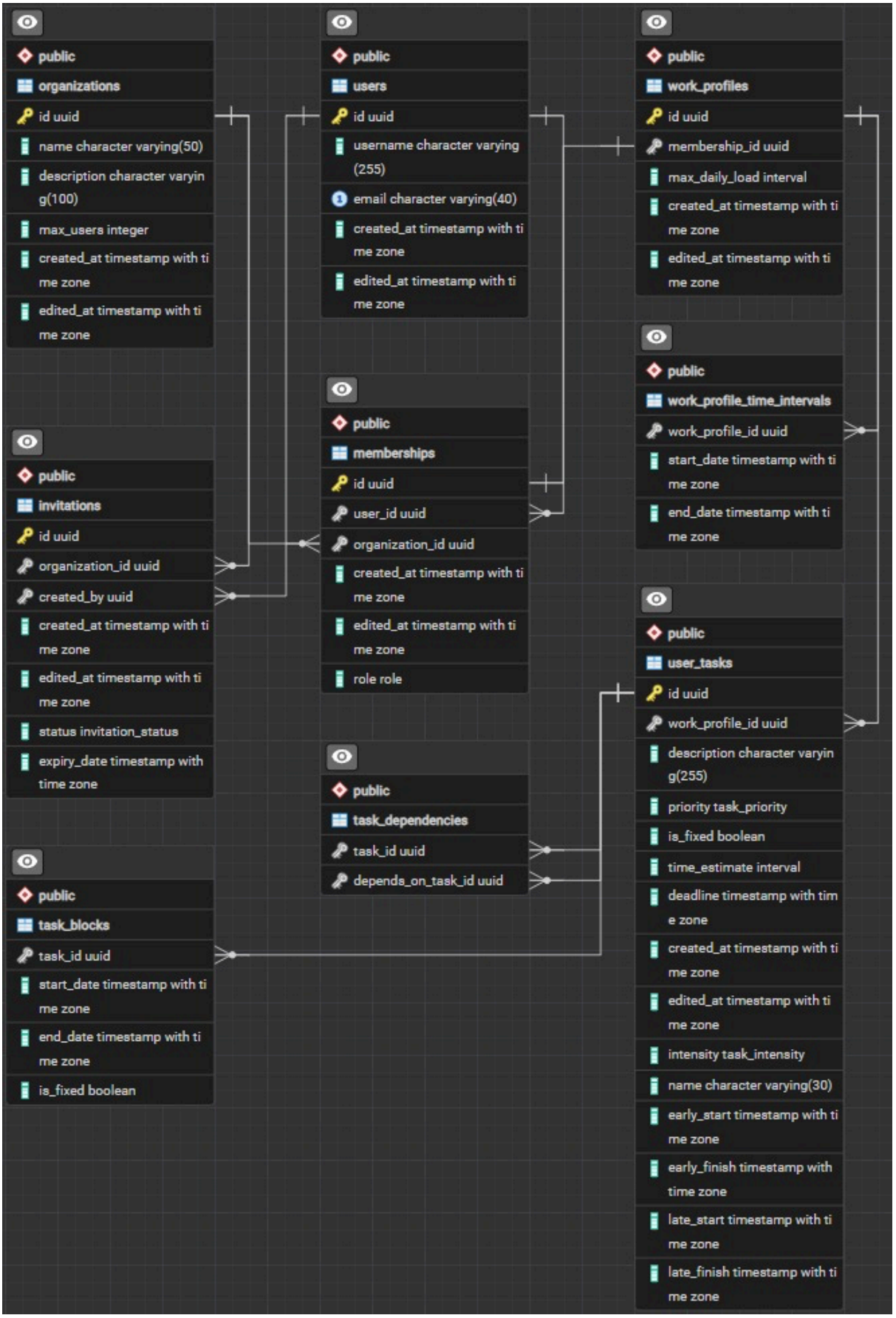
Über das IUserTaskPlanner-Interface wird eine entkoppelte Architektur realisiert, die es dem SchedulingAlgorithm erlaubt, komplexe Abhängigkeiten unter Berücksichtigung von Intensitätsprofilen und zeitlichen Puffern in einen konkreten WorkPlan zu überführen .



Die Kardinalitäten der Beziehungen sind im Diagramm dargestellt und zeigen die jeweiligen Abhängigkeiten zwischen den Entitäten.

3.2.1 ER-Schema der Datenbank

Die folgende Abbildung zeigt das Entity-Relationship-Schema der zugrunde liegenden Datenbank (lokale Entwicklungsumgebung).
Während das UML-Klassendiagramm die fachlichen Objekte und deren Beziehungen modelliert, stellt das ER-Schema die konkrete relationale Struktur der Datenbank dar.
Die Entitäten werden als Tabellen abgebildet, Beziehungen werden über Fremdschlüssel realisiert.
Beispielsweise wird die Beziehung zwischen Benutzer und Organisation über die Tabelle „Memberships“ modelliert.
Das ER-Schema dient als Grundlage für die Persistenz der im Klassendiagramm definierten Objekte und stellt die Konsistenz zwischen fachlichem Modell und technischer Umsetzung sicher.


public
organizations
id uuid
name character varying(50)
description character varyin g(100)
max_users integer
created_at timestamp with ti me zone
edited_at timestamp with ti me zone
public
users
id uuid
username character varying (255)
email character varying(40)
created_at timestamp with ti me zone
edited_at timestamp with ti me zone
public
work_profiles
id uuid
membership_id uuid
max_daily_load interval
created_at timestamp with ti me zone
edited_at timestamp with ti me zone
public
work_profile_time_intervals
work_profile_id uuid
start_date timestamp with ti me zone
end_date timestamp with ti me zone
public
invitations
id uuid
organization_id uuid
created_by uuid
created_at timestamp with ti me zone
edited_at timestamp with ti me zone
status invitation_status
expiry_date timestamp with time zone
public
memberships
id uuid
user_id uuid
organization_id uuid
created_at timestamp with ti me zone
edited_at timestamp with ti me zone
role role
public
user_tasks
id uuid
work_profile_id uuid
description character varyin g(255)
priority task_priority
is_fixed boolean
time_estimate interval
deadline timestamp with tim e zone
created_at timestamp with ti me zone
edited_at timestamp with ti me zone
intensity task_intensity
name character varying(30)
early_start timestamp with ti me zone
early_finish timestamp with time zone
late_start timestamp with ti me zone
late_finish timestamp with ti me zone
public
task_dependencies
task_id uuid
depends_on_task_id uuid
public
task_blocks
task_id uuid
start_date timestamp with ti me zone
end_date timestamp with ti me zone
is_fixed boolean

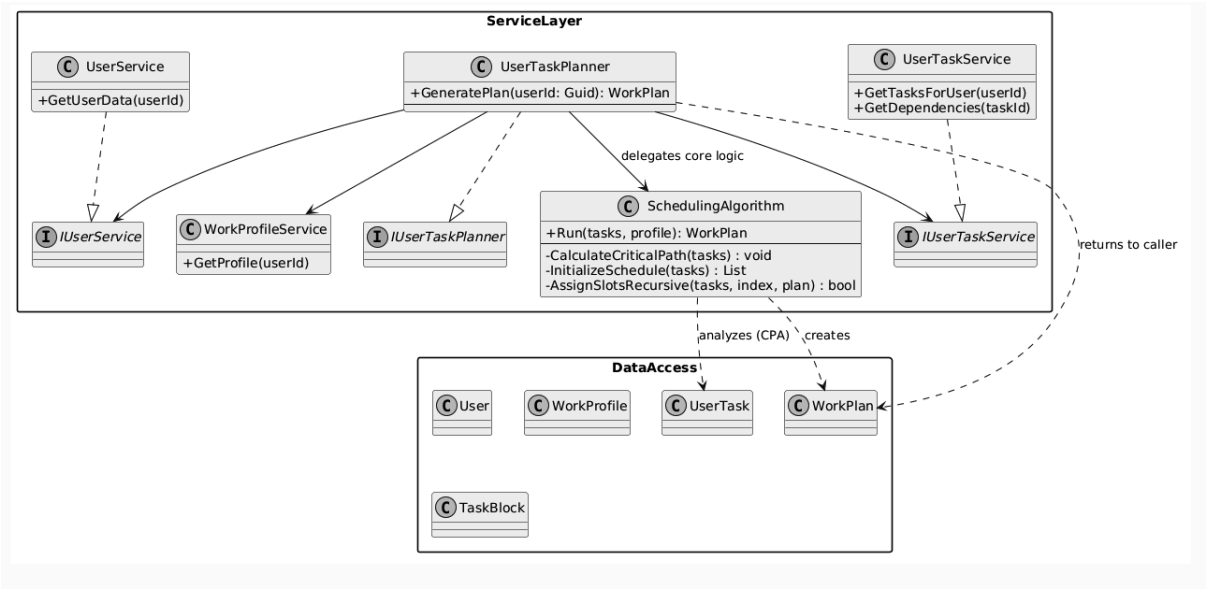
3.3 Architektur / Services

Während das statische Modell die Struktur der Daten definiert, beschreibt der Service Layer die funktionale Logik des Systems. Dieser Aufbau ist notwendig, um die Geschäftslogik strikt von der Datenhaltung zu trennen (Separation of Concerns).

Der `UserTaskPlanner` fungiert hierbei als zentrale Orchestrierungskomponente: Er ist dafür verantwortlich, die notwendigen Entitäten (Aufgaben, Abhängigkeiten und Nutzerprofile) über die jeweiligen Services zu laden und das Ergebnis der Planung in einem `WorkPlan`-DTO für das Frontend bereitzustellen.

Die eigentliche Intelligenz ist in den `SchedulingAlgorithm` ausgelagert. Diese Trennung ist entscheidend, da der komplexe, rekursive Algorithmus so isoliert von der Datenbanklogik getestet und optimiert werden kann. Die algorithmische Umsetzung erfolgt dabei in drei aufeinanderfolgenden Phasen:

1. **Analyse-Phase (CPA):** Mathematische Identifikation von Prioritäten und Engpässen (Kritischer Pfad).
2. **Initialisierungs-Phase:** Abgleich der Aufgaben mit den realen Zeitfenstern des Arbeitsprofils.
3. **Rekursive Planungs-Phase:** Effiziente Suche nach validen Zeitslots mittels Tiefensuche. Durch das im Code verankerte **implizite Backtracking** (Rekursion) wird sichergestellt, dass das System auch bei hoher Auslastung eine valide Lösung findet oder alternative Pfade prüft, ohne dass eine manuelle Konfliktlösung durch den Nutzer nötig ist.



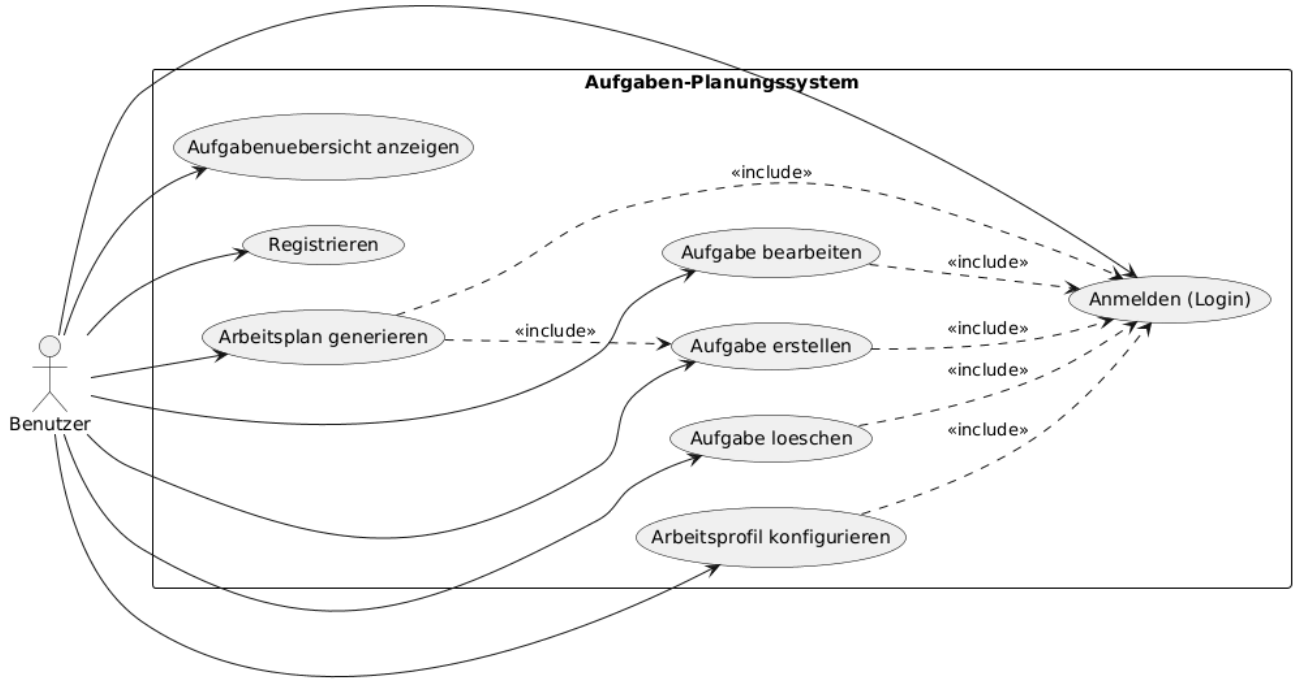
4. Dynamisches Modell

Das dynamische Modell beschreibt, wie die Klassen zur Laufzeit interagieren. Es wird anhand der vier wichtigsten Anwendungsfälle des MVP dargestellt.

4.1 Use-Case-Diagramm

Der zentrale Akteur ist der registrierte Benutzer. Der System-Rand umfasst das gesamte Aufgaben-Planungssystem.

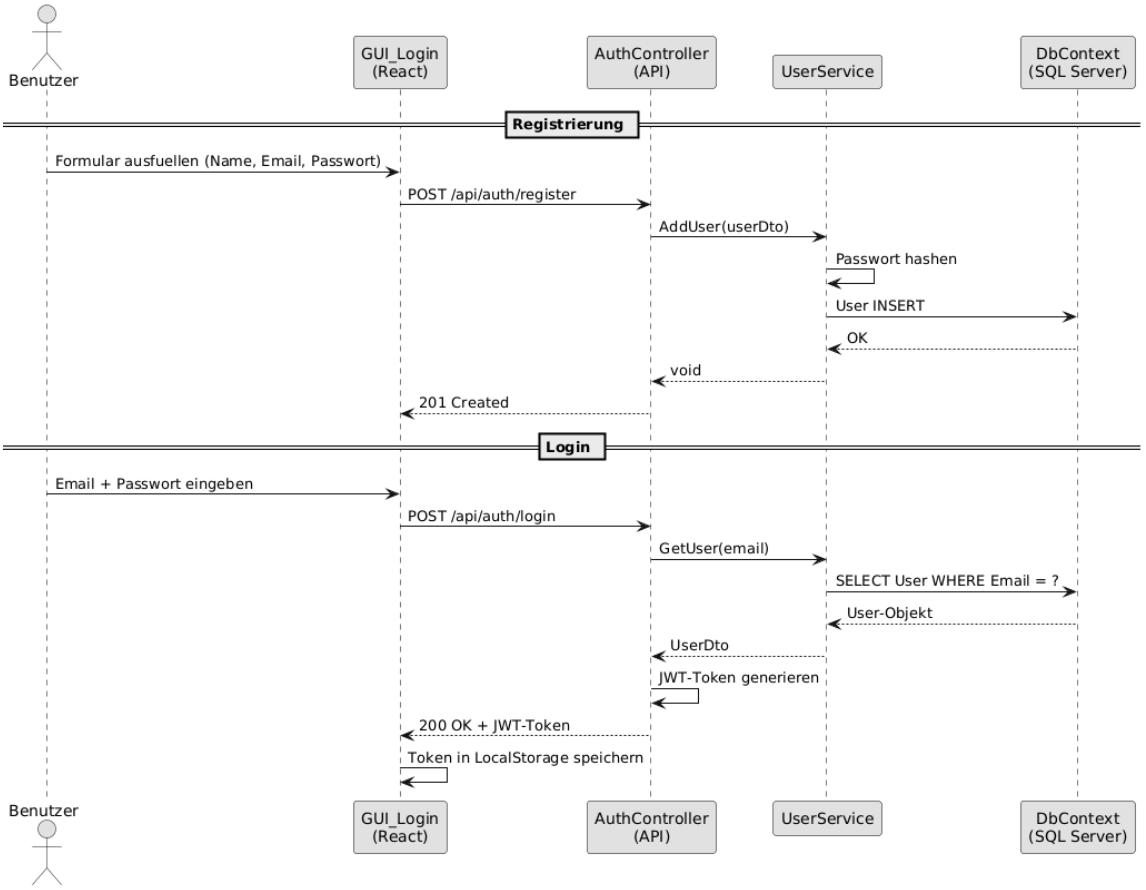
•**UC1:** Registrieren
•**UC2:** Anmelden (Login)
•**UC3:** Aufgabe erstellen
•**UC4:** Aufgabe bearbeiten
•**UC5:** Aufgabe löschen
•**UC6:** Arbeitsprofil konfigurieren
•**UC7:** Arbeitsplan generieren
•**UC8:** Aufgabenübersicht anzeigen



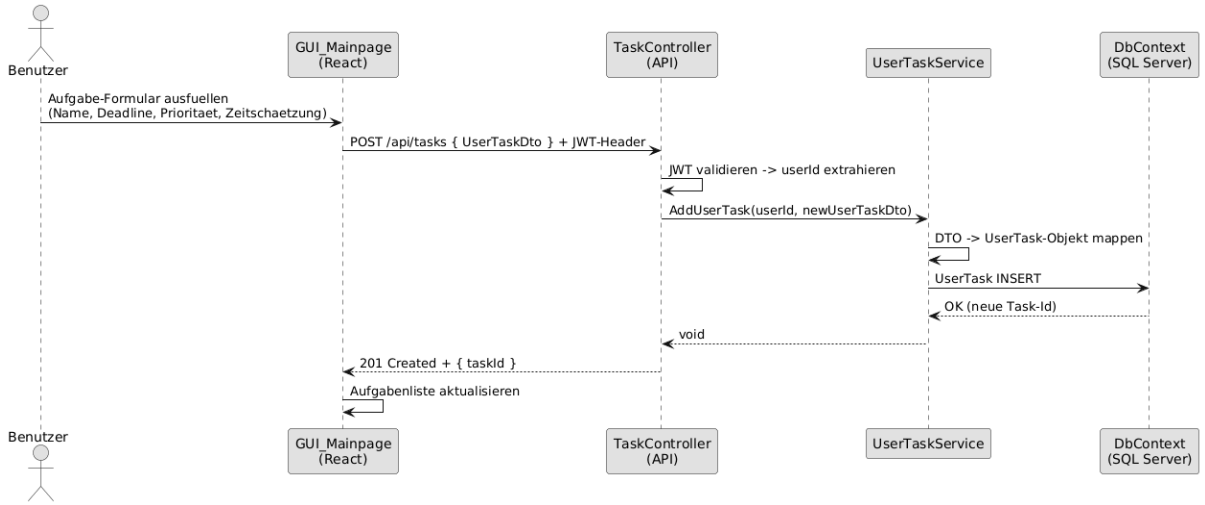
Begründung: UC7 (Arbeitsplan generieren) schließt UC3 ein, da ohne vorhandene Aufgaben kein Plan generiert werden kann. Alle Aktionen nach UC1 erfordern einen erfolgreichen Login (UC2).

4.2 Sequenzdiagramm: Benutzer registrieren und anmelden

Dieser Ablauf zeigt die Interaktion bei der Erstregistrierung und dem anschließenden Login.

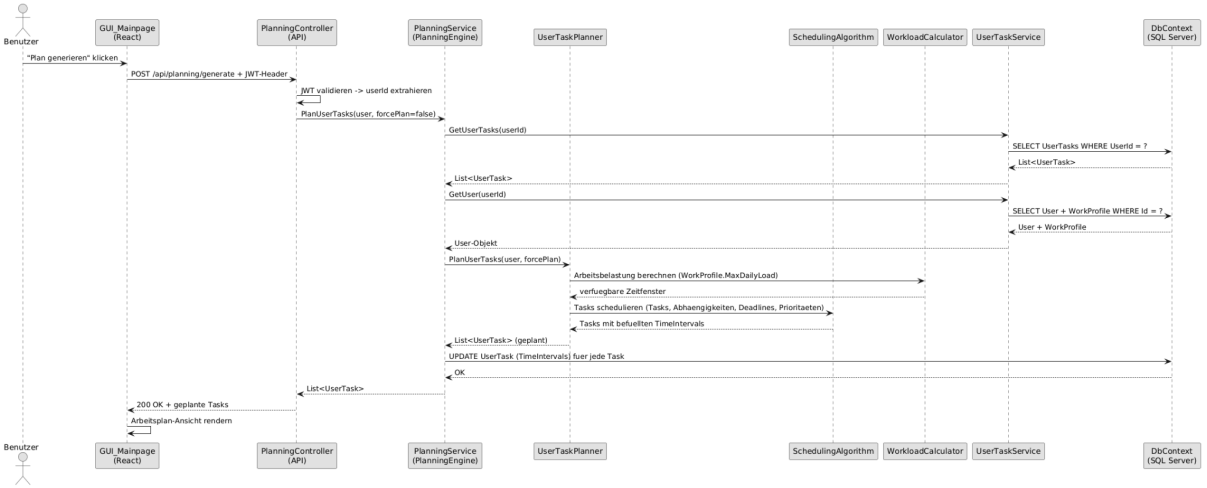


4.3 Sequenzdiagramm: Aufgabe erstellen



4.4 Sequenzdiagramm: Arbeitsplan generieren

Dies ist der komplexeste und für das System charakteristische Ablauf.



Begründung der Methoden:
Aus diesem Szenario ergeben sich folgende Methoden, die den Klassen zugewiesen werden:

- `PlanningController.GeneratePlan()` – delegiert an Service
- `PlanningService` (via `IUserTaskPlanner`) → `PlanUserTasks(user: User, forcePlan: bool): List<UserTask>`
- `SchedulingAlgorithm`: weist Aufgaben konkrete `TimeInterval`-Slots zu (Reihenfolge nach Priorität, Deadline, Abhängigkeiten)

4.4.1 Beschreibung des Planungsalgorithmus

Dieser Abschnitt beschreibt den Algorithmus zur automatischen Generierung eines Arbeitsplans (SchedulingAlgorithm). Die Darstellung folgt der tatsächlichen internen Logik des Systems und ist in klar getrennte Phasen unterteilt.

Ziel des Algorithmus ist es, Aufgaben unter Berücksichtigung von Abhängigkeiten, Prioritäten, Deadlines, Intensität sowie individueller Arbeitsprofile realistisch und effizient in verfügbare Zeitfenster einzuplanen.

1. Analyse der Aufgaben und kritischer Pfad
   Zu Beginn werden alle Aufgaben und ihre Abhängigkeiten als gerichteter Graph betrachtet. Dabei wird zunächst geprüft, ob zyklische Abhängigkeiten vorliegen, da in diesem Fall keine gültige Planung möglich ist.

Anschließend erfolgt eine topologische Sortierung der Aufgaben. Auf dieser Grundlage werden frühestmögliche Start- und Endzeitpunkte berechnet. Diese Analyse dient insbesondere dazu, kritische Aufgabenketten zu identifizieren und frühzeitig zu erkennen, ob Deadlines prinzipiell eingehalten werden können.

Falls bereits in dieser Phase festgestellt wird, dass eine konsistente Planung aufgrund von Abhängigkeiten oder Deadlines nicht möglich ist, wird der Planungsprozess abgebrochen und ein entsprechender Hinweis erzeugt.

2. Initialisierung des Planungsraums
   Nach der erfolgreichen Analyse werden alle relevanten Daten geladen. Dazu gehören insbesondere die offenen Aufgaben mit ihren Attributen (Restdauer, Priorität, Deadline, Intensität), das Arbeitsprofil der Nutzenden (Arbeitszeiten, maximale tägliche Belastung) sowie vorhandene feste Termine.

Auf Basis dieser Informationen werden Arbeitszeitfenster erzeugt und feste Termine als Blockierungen eingetragen. Dadurch entsteht eine strukturierte Darstellung aller freien Zeitslots, die für die Planung verwendet werden können.

Zusätzlich werden interne Zustände initialisiert, insbesondere die verbleibende Restdauer der Aufgaben sowie Hilfsvariablen zur Steuerung der Planung.

3. Rekursive Planung der Aufgaben
   Die eigentliche Einplanung erfolgt rekursiv. In jedem Schritt wird eine geeignete Aufgabe ausgewählt. Dabei werden insbesondere folgende Kriterien berücksichtigt:

- Einhaltung von Abhängigkeiten (eine Aufgabe kann erst geplant werden, wenn alle Vorgänger abgeschlossen sind)
- Deadlines
- Priorität
- Intensität

Für die ausgewählte Aufgabe wird ein geeigneter freier Zeitslot gesucht. Dabei wird geprüft, ob die Aufgabe vollständig oder teilweise in den Slot eingeplant werden kann.
Ist der Slot kleiner als die verbleibende Restdauer der Aufgabe, wird ein Teilblock geplant und die Restdauer entsprechend reduziert. Dieser Mechanismus ermöglicht eine flexible Aufteilung von Aufgaben über mehrere Zeitintervalle hinweg.
Nach erfolgreicher Einplanung wird der Algorithmus mit dem aktualisierten Zustand erneut aufgerufen, bis alle Aufgaben verarbeitet wurden.

4. Behandlung von Konflikten
   Falls für eine Aufgabe kein geeigneter Zeitslot gefunden wird, kehrt der Algorithmus in einen vorherigen Zustand zurück und versucht eine alternative Einplanung.

Dieses Verhalten ergibt sich direkt aus der rekursiven Struktur des Algorithmus. Ein separates Backtracking-System oder ein expliziter Stack zur Verwaltung von Zuständen ist nicht erforderlich.

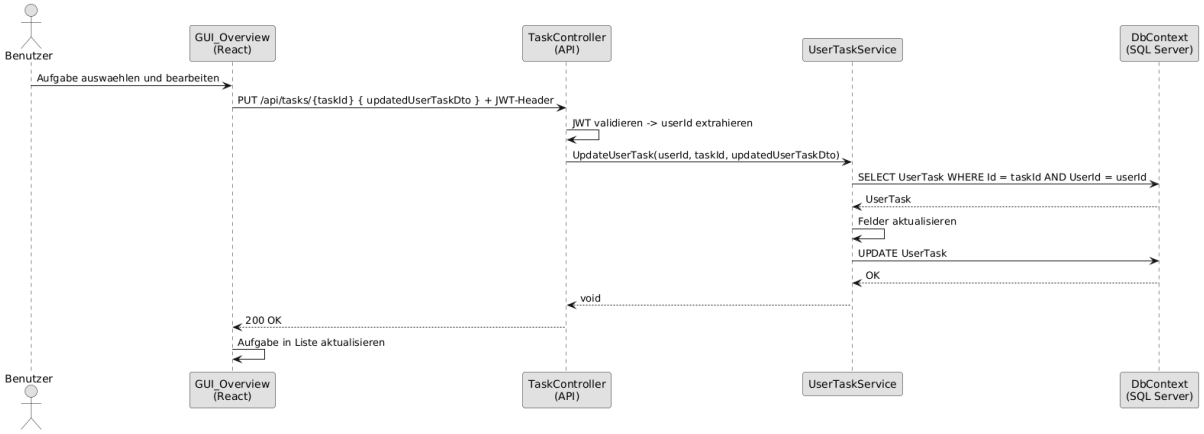
5. Abschluss und Validierung
   Nach Abschluss der Planung wird der erzeugte Arbeitsplan überprüft. Dabei wird insbesondere sichergestellt, dass:

- keine Zeitintervalle sich überschneiden
- Deadlines eingehalten werden
- definierte Arbeitszeitgrenzen nicht überschritten werden

Ist der Plan gültig, wird er gespeichert und an das Frontend übergeben. Andernfalls wird ein Konflikt gemeldet und die Planung entsprechend angepasst oder abgebrochen.

Zusammenfassend basiert der Planungsalgorithmus auf einer vorgelagerten Analyse der Aufgabenstruktur, einer klaren Initialisierung des Planungsraums sowie einer rekursiven Einplanung der Aufgaben. Dadurch wird eine flexible und gleichzeitig nachvollziehbare Planung ermöglicht, ohne ein separates Backtracking-Verfahren implementieren zu müssen.

4.5 Sequenzdiagramm: Aufgabe bearbeiten



4.6 Identifizierte Methoden je Klasse

Aus den Sequenzdiagrammen lassen sich folgende Methoden systematisch ableiten:

Klasse	Methode	Auslöser
`AuthController`	`LoginUser()`	Login-Sequenz
`AuthController`	`RegisterUser()`	Registrierungs-Sequenz
`TaskController`	`CreateTask()`	Aufgabe erstellen
`TaskController`	`GetTask()`	Aufgabenübersicht
`TaskController`	`UpdateTask()`	Aufgabe bearbeiten
`TaskController`	`DeleteTask()`	Aufgabe löschen
`PlanningController`	`GeneratePlan()`	Arbeitsplan generieren

`UserService`	`GetUser(userId)`	Login, Planung
`UserService`	`AddUser(userDto)`	Registrierung
`UserService`	`UpdateUser(userDto)`	Profil bearbeiten
`UserService`	`DeleteUser(userId)`	Konto löschen
`UserTaskService`	`GetUserTasks(userId)`	Aufgabenliste, Planung
`UserTaskService`	`AddUserTask(userId, dto)`	Aufgabe erstellen
`UserTaskService`	`UpdateUserTask(userId, taskId, dto)`	Aufgabe bearbeiten
`UserTaskService`	`UpdateUserTasks(userId, tasks)`	Plan speichern
`UserTaskService`	`DeleteUserTask(userId, taskId)`	Aufgabe löschen
`UserTaskPlanner`	`PlanUserTasks(user, forcePlan)`	Plan generieren
`SchedulingAlgorithm`	`GenerateSchedule(user)`	Aufgaben einplanen
WorkProfileService	`UpdateWorkProfile()`	Arbeitsprofil konfigurieren
PlanningService	`MoveTaskBlock()`	Arbeitsplan anpassen
OrganizationService	`RemoveMember()`	Nutzer aus Team löschen

5. API-Spezifikation

Methode	**Pfad**	**Auth**	**Input/Output DTOs**	**Status-Codes**
Get Org	organizations	OAuth	OrgId, OrgDTO	200, 400, 401, 404, 500

Get Orgs	organizations	OAuth	OrgIdList, OrgDTOList	200, 400, 401, 404, 500
Join Org	organizations/join	OAuth	InviteCode	200, 400, 401, 404, 500, 504
Leave Org	organization/leave	OAuth	OrgId	200, 400, 401, 404, 500, 504
CreateTask	tasks	OAuth	TaskDTO	201, 400, 404, 500,
GetTasks	tasks	OAuth	TaskIdList	200, 400, 401, 404, 500
DeleteTask	tasks	OAuth	TaskDTO	200, 400, 404, 500
UpdateTask	tasks	OAuth	TaskId, TaskDTO	200, 400, 404, 500
UpdateTasks	tasks	OAuth	TaskDTOList	200, 400, 401, 404, 500
ScheduleTasks	scheduler	OAuth	ScheduleDTO	200, 400, 401, 404, 500

6. Design Patterns

Das System setzt folgende Entwurfsmuster ein:

Pattern	Anwendungsort	Nutzen
Repository Pattern	`UserService`, `UserTaskService` greifen auf `DbContext` zu	Kapselt Datenbankzugriff, ermöglicht Austausch des Datenlayers

Dependency Injection	Alle Services erhalten `DbContext` und andere Abhängigkeiten per Konstruktor	Modularität, Testbarkeit (Mock-Objekte)
Service Layer Pattern	`UserService`, `UserTaskService`, `PlanningService`	Trennt Geschäftslogik von Controller und Datenschicht
Strategy Pattern	`IUserTaskPlanner` / `SchedulingAlgorithm`	Erlaubt Austausch des Planungsalgorithmus ohne API-Änderung
DTO Pattern	`UserDto`, `UserTaskDto`	Verhindert, dass Datenbankentitäten direkt exponiert werden

7. Frontend-Struktur

Die React-Anwendung gliedert sich in folgende Hauptkomponenten:

Komponente	Zuständigkeit
`GUI_Login`	Formular für Login und Registrierung
`GUI_Mainpage`	Navigations-Hub mit Buttons zu allen Unterbereichen
`GUI_Overview`	Aufgabenübersicht (Deadlines, anstehende Tasks, Wochenmengen)
`GUI_ProfileButton`	Profilbild, Name, Signed-In-Status
`GUI_MyTeams`	Liste aller Teams denen man zugehört
`GUI_Planner`	Kalenderübersicht der Aufgaben und Aufgabenverwaltung
`GUI_Tasks`	Listenübersicht der Aufgaben und Aufgabenverwaltung
`GUI_Settings`	Seite der Profileinstellungen

Der API-Zugriff erfolgt über einen zentralen **API-Service** im Frontend, der JWT-Token automatisch in den `Authorization`-Header einfügt. State wird per **React Context API** oder Redux Toolkit verwaltet (User State, Task State, Plan State).

8. Scope der ersten Iteration (MVP)

Funktion	Status	Begründung
F20 Registrierung & Login	**Go**	Basis Voraussetzung für alle weiteren Funktionen
F10 Aufgabe konfigurieren (CRUD)	**Go**	Kern Datengrundlage für die Planung
F50 Arbeitsprofil konfigurieren	**Go**	Pflicht Input für den Planungsalgorithmus
F40 Arbeitsplan generieren	**Go**	Kernfunktionalität des Produkts
F30 Organisierende einladen	**No-Go**	Komplexe Berechtigungen Logik
F60 Transparente Planung	**No-Go**	Erweiterte Visualisierung

Das Pflichtenheft beschreibt den vollständigen fachlichen Zielumfang. In der ersten Iteration dieses Designmodells werden nur die Kernfunktionen Registrierung, Aufgabenverwaltung, Arbeitsprofil und Arbeitsplangenerierung detailliert modelliert. Organisationsverwaltung und Einladungskontingente werden, wie in der Tabelle festgelegt, in späteren Iterationen erweitert.

9. Deployment und Entwicklungsinfrastruktur

Die Bereitstellung erfolgt via **Docker-Container** auf einem VPS oder Cloud-Anbieter (Azure/AWS). Die CI/CD-Pipeline (GitHub Actions) automatisiert Build, Test und Deployment:

1. Developer Push → GitHub Actions ausgelöst
2. Unit Tests (MSTest) + Integration Tests → müssen bestehen
3. UI Tests (Playwright) → müssen bestehen
4. Code-Coverage-Prüfung → Mindestwert 80%
5. Docker-Image bauen und in Registry pushen
6. Automatisches Deployment bei Merge in `main`

Versionierung erfolgt nach **Semantic Versioning** (MAJOR.MINOR.PATCH). Bei kritischen Fehlern nach Deployment kann auf das letzte stabile Docker-Image zurückgerollt werden.